

PERSONAL OS

System Analysis and Technical Documentation

Project: Personal OS productivity and AI-automation dashboard

Deployed project: personal-os-git-main-florah-s-projects.vercel.app

Prepared from the supplied Personal OS source archive and project configuration files.

1. Project Overview

Personal OS is a personal productivity and AI-automation framework designed to give one user a central place to manage tasks, persistent knowledge, projects, and recurring automations. The project combines a browser-based dashboard with a local Markdown-based knowledge system and an AI-agent layer implemented around Claude Code and external integrations.

The deployed dashboard is primarily a front-end interface. It provides two main areas: Tasks and Memory. The Tasks area supports a Kanban-style board and a list view, while the Memory area reads structured Markdown files such as CLAUDE.md and files inside the memory directory. The dashboard uses the browser File System Access API to open and save local files rather than relying on a traditional application database.

Main objectives

- Provide a single interface for planning and tracking work.
- Keep important personal/project context in persistent Markdown files.
- Allow tasks to be created, edited, completed, reordered, and moved between sections.
- Support an AI-agent workflow that can read the user's context and perform repeatable automations.
- Synchronize selected automation outputs with services such as Notion, Gmail, Google Calendar and web research tools.
- Maintain a local, human-readable knowledge vault that can also be opened in Obsidian.

2. Problem Statement

Users who manage multiple projects and information sources often have their work spread across task lists, email, calendars, notes, project documents and external productivity platforms. This fragmentation makes it difficult to maintain a consistent view of priorities, remember useful context, and repeat routine information-processing tasks.

The Personal OS project addresses this problem by combining task management, persistent Markdown-based memory, project records and AI-powered automations into one workflow. Instead of treating tasks and knowledge as separate systems, the project uses the vault as a persistent knowledge layer and the AI agent as an orchestration layer.

Specific problems addressed

- Fragmented information: useful context may exist in different applications and files.
- Loss of context between sessions: important decisions and project information need persistent storage.
- Manual repetitive work: daily briefs, project tracking and market monitoring can require repeated data gathering.
- Weak connection between tasks and knowledge: a task often needs background information stored elsewhere.
- Difficulty maintaining historical records: reports, standups, market scans and project updates need an archive.
- Inconsistent workflows: automation specifications and operating rules need to be stored in a predictable structure.

3. System Scope and Major Features

Component	Purpose	Key Functions
-----------	---------	---------------

Task Management	Manage work items	Create, edit, delete, complete, reorder and move tasks; add notes and subtasks.
Board View	Visual task planning	Kanban-style sections with drag-and-drop tasks and sections.
List View	Compact task planning	Display tasks as a structured list and support quick task entry.
Memory	Persistent context	Read and display CLAUDE.md, memory files and memory directories; search and edit supported files.
Vault	Long-term knowledge	Store people, business, project, research, meeting and historical information in Markdown.
Automations	Reduce repetitive work	Sprint Tracker, Morning Brief and Market Pulse are specified/built; more automations are planned.
Scheduler	Recurring execution	Stores schedules for recurring commands such as Market Pulse, Morning Brief and Sprint Tracker.
External integrations	Gather/sync information	Notion, Gmail, Google Calendar and Chrome/web research through the agent/MCP workflow.

4. System Architecture

The system is best understood as a layered architecture. The deployed dashboard is a static web front end, while the Personal OS framework adds an AI-agent/orchestration layer and a persistent file-based knowledge layer.

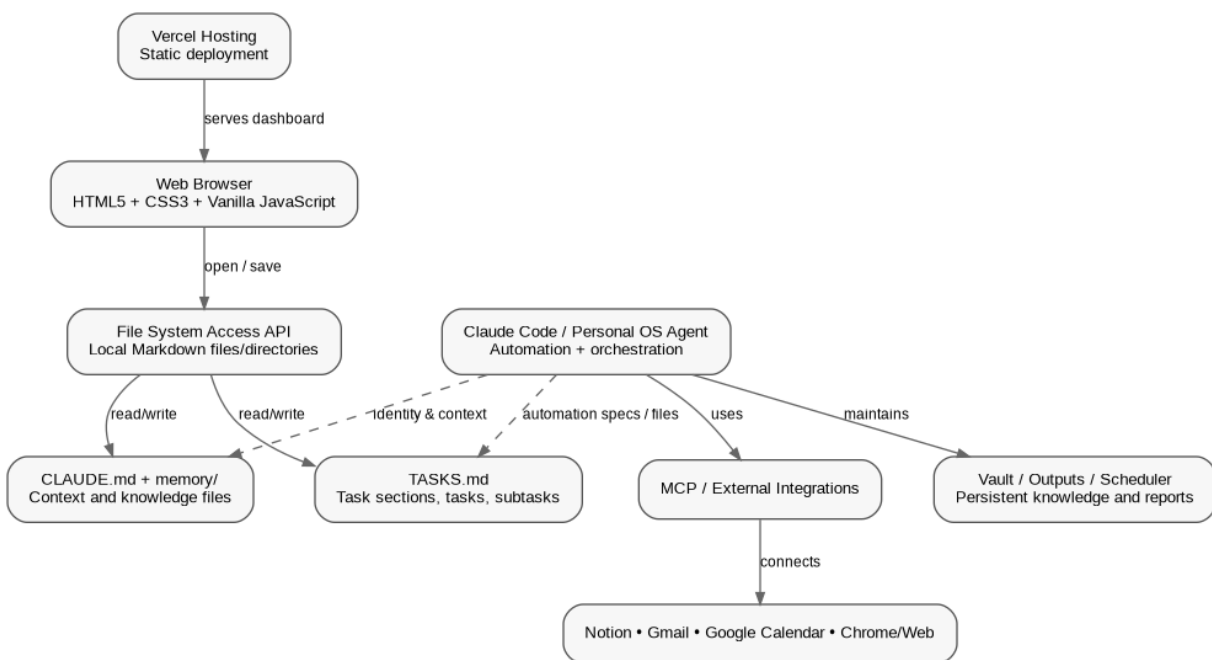


Figure 1. High-level Personal OS system architecture.

4.1 Presentation layer

The dashboard is implemented as a single HTML document containing the interface, CSS styling and JavaScript logic. The interface provides navigation between Tasks and Memory, plus Board/List task views.

4.2 Local data layer

Tasks are stored as Markdown in TASKS.md. The dashboard parses the Markdown into in-memory JavaScript objects, renders the interface, and converts the updated task state back to Markdown when saving. The Memory section similarly reads CLAUDE.md and Markdown files under the memory directory.

4.3 AI orchestration layer

Claude Code acts as the agent/orchestration environment. The project defines slash commands such as /setup, /ingest, /status, /sprint-tracker, /morning-brief and /market-pulse. The agent follows the project's CLAUDE.md rules to maintain the vault, execute automations and record outputs.

4.4 Integration layer

The automation specifications describe integrations with Notion, Gmail, Google Calendar and Chrome/web research. These integrations are used by the agent to gather information, create records, generate reports and update project state.

4.5 Hosting

The supplied project is deployed through Vercel. The deployed URL is the public-facing entry point, while the dashboard itself is a static HTML application. The supplied archive also contains a package.json with Node/npm metadata and dotenv as a dependency, although the dashboard's core interaction logic is implemented in browser-side JavaScript.

5. Main Data Flow

1. The user opens the Personal OS dashboard in a supported browser.
2. The user selects a Markdown task file using the browser's File System Access API.
3. JavaScript parses headings, tasks, checkboxes, notes and subtasks into application state.
4. The interface renders the task state as a Board or List view.
5. When changes are made, the application automatically saves the updated Markdown after a short debounce period.
6. The Memory section can open a Personal OS directory and read CLAUDE.md plus Markdown files in memory subdirectories.
7. Separately, the Claude Code agent reads the project rules, vault and automation specifications.
8. Automations gather information from connected services and write reports, history and knowledge back into the vault and/or Notion.

6. Frameworks and Technologies Used

Technology / Framework	Role	Evidence in project
HTML5	Dashboard structure	dashboard.html contains the application markup.
CSS3	User interface and responsive styling	Inline CSS defines dashboard layout, cards, columns, buttons, modals and views.
Vanilla JavaScript	Client-side application logic	dashboard.html contains JavaScript for state, parsing,

		rendering, drag-and-drop and file operations.
File System Access API	Local persistence	showOpenFilePicker(), showDirectoryPicker(), getFile(), createWritable() and file watching are used.
Markdown	Human-readable storage	TASKS.md, CLAUDE.md, vault pages, memory files and automation specifications use Markdown.
Vercel	Web deployment/hosting	The supplied application is deployed on a Vercel URL.
Node.js / npm	Project/package management	package.json and package-lock.json are included; dotenv is listed as a dependency.
Claude Code	AI agent and orchestration	CLAUDE.md and project slash commands define the agent workflow.
MCP integrations	External service connectivity	Project specifications reference Notion, Gmail, Google Calendar and Chrome MCP workflows.
Obsidian	Knowledge-base interface	Getting Started documentation identifies Obsidian as the intended interface for the persistent vault.
Notion	Progress/output database	Automation specifications define databases for sprint tracking, daily briefs and market scans.
Gmail / Google Calendar	Information sources	Morning Brief is specified to pull unread Gmail threads and today's Calendar events.

Important implementation distinction: the browser dashboard does not itself call Gmail, Calendar or Notion APIs. Those services belong to the broader Personal OS automation ecosystem and are accessed by the Claude Code/MCP layer according to the project specifications.

7. Use Case Diagram

The main actor is the User. The dashboard supports task and memory operations, while the AI agent interacts with external services and the persistent file system. The diagram therefore shows both direct dashboard use cases and the supporting automation ecosystem.

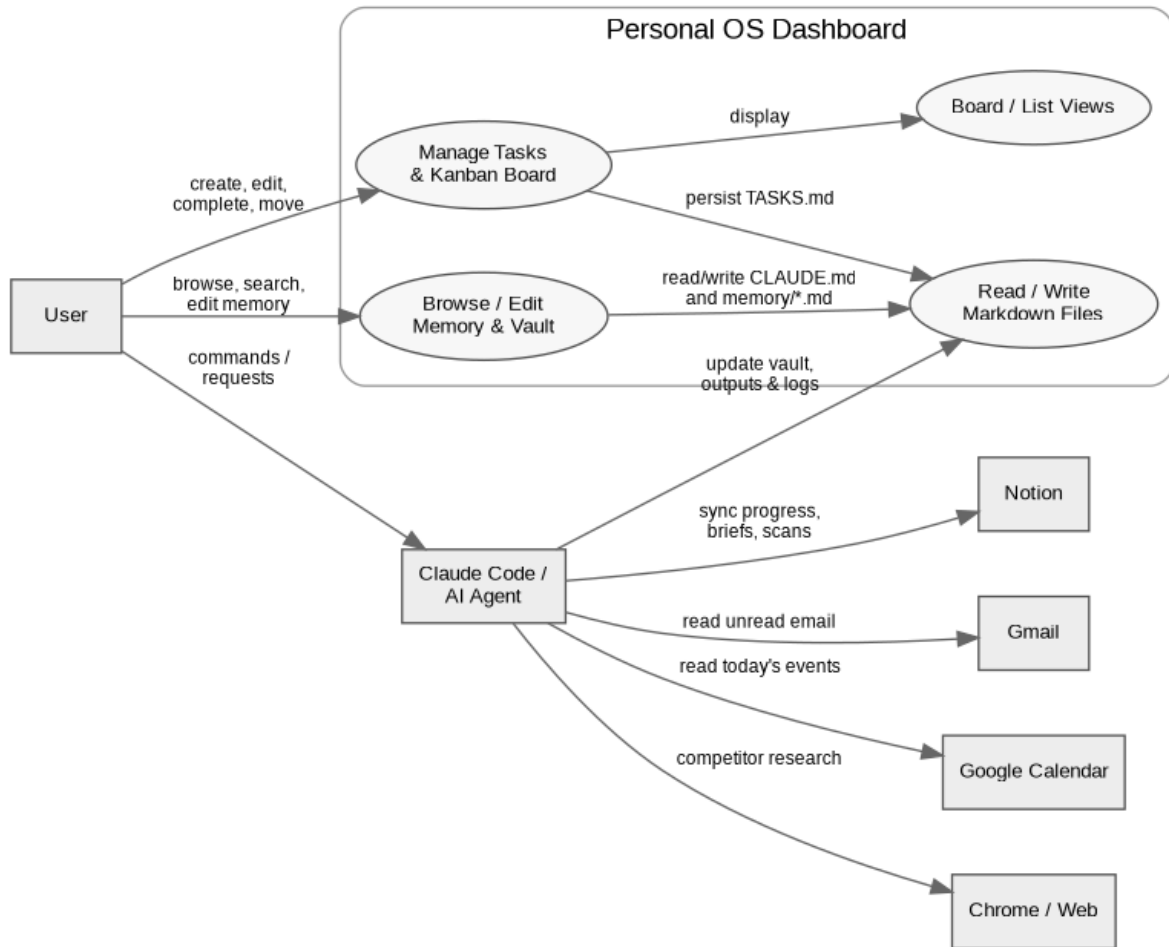


Figure 2. Personal OS use case diagram.

Use case descriptions

Use Case	Description
Manage Tasks	User creates, edits, completes, deletes and organizes tasks.
Switch Board/List View	User changes how tasks are displayed.
Move Tasks/Sections	User uses drag-and-drop to reorder tasks or move them between sections.
Open Task File	User selects a Markdown task file from the local file system.
Save Task File	System writes updated task state back to Markdown.
Browse Memory	User opens a Personal OS directory and browses CLAUDE.md and memory files.
Edit Memory	User can edit supported Markdown memory/configuration content.
Run Automation	User invokes a Personal OS command through the AI-agent environment.
Sync with Notion	Automation reads/writes project progress or generated reports to Notion.
Read Gmail/Calendar	Morning Brief automation gathers relevant email and calendar context.

Perform Web Research	Market Pulse uses browser/web research to gather competitor intelligence.
----------------------	---

8. Current and Planned Automations

The supplied project defines ten planned automation areas. Three are marked Done in TASKS.md: Sprint Tracker, Morning Brief and Market Pulse. The remaining automations are listed as future work.

Automation	Purpose	Status
Sprint Tracker	Reads the Notion progress board, generates standups and tracks velocity.	Done
Morning Brief	Combines Gmail, Google Calendar and Notion context into a daily brief.	Done
Market Pulse	Monitors competitor websites, web news and internal mentions, then produces a tagged market report.	Done
Research Team	Planned research automation.	Todo
Personal CRM	Planned contact/relationship automation.	Todo
Meeting Intel	Planned meeting preparation/intelligence automation.	Todo
Email Triage	Planned email classification/processing automation.	Todo
Expense Wrangler	Planned expense/receipt automation.	Todo
Content Machine	Planned content production automation.	Todo
Weekly Exec Report	Planned weekly personal review/reporting automation.	Todo

9. Technical Characteristics, Strengths and Limitations

Technical characteristics

- Human-readable storage: Markdown files can be opened and edited outside the dashboard.
- Low infrastructure overhead: the dashboard does not require a conventional application database for task storage.
- Local-first interaction: the browser can work directly with files selected by the user.
- AI-oriented architecture: project rules, memory and automation specifications are designed for agent execution.
- Modular automation design: each automation has a dedicated specification and output/history structure.

Limitations / considerations

- The dashboard depends on browser support for the File System Access API.
- Because data is file-based, the user must grant file/directory access to the browser.
- The static dashboard alone is not a complete cloud multi-user application.
- External integrations depend on the configured Claude Code/MCP environment and authentication.

- The supplied soul.md is still a template, and the project notes indicate that /setup has not been completed.
- The project status files indicate that Notion bootstrap/configuration is still pending for the supplied setup.

10. Conclusion

Personal OS is a hybrid productivity system rather than only a task-management website. Its dashboard provides the user interface, Markdown provides portable persistent storage, the vault provides long-term knowledge management, and Claude Code provides AI-driven orchestration and automation. The architecture is therefore designed around a human-readable local knowledge base connected to external productivity services when automation is required.

For an academic/software-engineering project description, the central system concept can be summarized as: a browser-based personal productivity dashboard backed by Markdown files and extended by an AI-agent automation framework.

11. Source Basis

This report was generated from the supplied personalOS.zip project archive, including dashboard.html, CLAUDE.md, GETTING-STARTED.md, TASKS.md, soul.md, vault files, memory files, scheduler configuration and the specifications for Sprint Tracker, Morning Brief and Market Pulse. The supplied deployment URL was also reviewed as the project reference; automated web access to that URL redirected to a Vercel authentication endpoint, so the detailed implementation analysis in this report is grounded in the supplied source archive.